

SYSTEM DESIGN DOCUMENT

Magizhthunnu

Complete Architecture & Engineering Blueprint
for a Full-Stack Online Food Delivery Platform

VERSION

1.0.0

DATE

March 2026

ROLE

Senior Full Stack Architect

STACK

MERN (MongoDB)

STATUS

Greenfield

React 18

TypeScript 5.3

Node.js 22

Express.js

MongoDB Atlas

Mongoose 8

Redis

BullMQ

Paytm Gateway

Socket.IO

Docker

GitHub Actions

Table of Contents

1	Executive Summary	2	High-Level Architecture (HLD)
3	Technology Stack Decisions	4	Frontend Architecture
5	Backend Architecture	6	Database Design (LLD)
7	REST API Design	8	Authentication & RBAC
9	Payment Architecture (Paytm)	10	Real-Time Order Tracking
11	Security Architecture	12	Deployment Architecture
13	Scalability Design	14	Testing Strategy
15	Complete Folder Structure		

SECTION 01

Executive Summary

Business context, goals, and stakeholders

Magizhthunnu (meaning "delightful" in Tamil) is a full-stack online food delivery platform connecting customers with local restaurants. Built on the MERN stack with TypeScript, it enables customers to browse restaurants, place orders, and pay digitally — while giving restaurants a management dashboard and admins a control plane.



Business Goals

- Connect customers & local restaurants
- Frictionless digital ordering experience
- Integrated Paytm payment processing
- Real-time order tracking
- Admin & restaurant management dashboards



User Roles

- **Customer** — Browse, order, track, review
- **Restaurant Owner** — Manage menu, orders, hours
- **Admin** — Platform oversight, analytics



Core Features

- Restaurant discovery (geolocation)
- Dynamic menu with customizations
- Shopping cart with TTL auto-clear
- Multi-step checkout & Paytm payments



Non-Functional Requirements

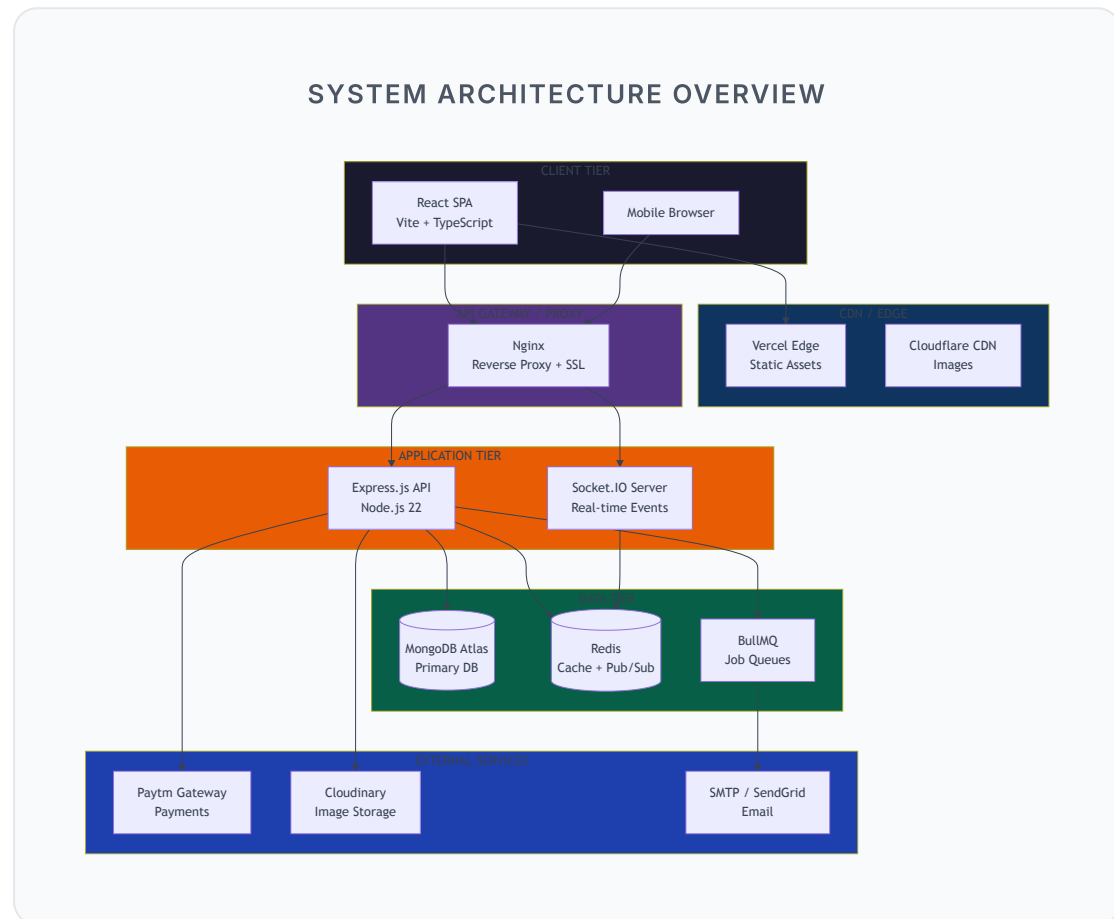
- Response time < 200ms (cached)
- 99.9% uptime SLA
- HTTPS + JWT security
- Horizontal scalability
- 80%+ test coverage

- Live order status (Socket.IO)
- Ratings & reviews system

SECTION 02

High-Level Architecture

3-tier system architecture and component interactions



Component Responsibilities

COMPONENT	TECHNOLOGY	RESPONSIBILITY	HOST
React SPA	React 18 + Vite	UI rendering, routing, state management	Vercel
Nginx	Nginx 1.25	SSL termination, reverse proxy, rate limiting	AWS EC2 / Render
Express API	Node.js 22 + Express	Business logic, REST endpoints, auth	Render / Railway
Socket.IO	Socket.IO 4.x	Real-time order status push	Same as API
MongoDB Atlas	MongoDB 7.x	Persistent data store (all collections)	MongoDB Atlas
Redis	Redis 7.x	Caching, session store, Pub/Sub	Redis Cloud / Upstash
BullMQ	BullMQ + Redis	Async jobs: emails, notifications, reports	Same as API
Cloudinary	Cloudinary SDK	Restaurant/menu image storage + CDN	Cloudinary Cloud
Paytm Gateway	Paytm Node SDK	Payment initiation, checksum, webhook	Paytm Cloud

SECTION 03

Technology Stack

Full decision table with version and rationale



Database Decision: MongoDB over PostgreSQL

Menu items vary wildly across restaurants (flexible schema). Nested documents map naturally to cart items, order line items, and embedded delivery addresses. JSON-native eliminates impedance mismatch with Node.js/TypeScript. Built-in TTL indexes handle cart expiry. Horizontal sharding for future scale. PostgreSQL would win for strict financial auditing — MongoDB wins here for agility.

Frontend Stack

LAYER	TECHNOLOGY	VERSION	REASON
UI Framework	React	18.3	Component model, concurrent features, wide ecosystem
Build Tool	Vite	5.x	Sub-second HMR, optimized production bundles
Language	TypeScript	5.3	Strict type safety, better DX, catches bugs at compile time
Styling	Tailwind CSS	3.x	Utility-first, no CSS file bloat, consistent design system
Routing	React Router	6.x	File-based routing with data loaders
HTTP Client	Axios	1.x	Request/response interceptors for auth tokens
State	Context API + useReducer	Built-in	Sufficient for Auth + Cart; avoids Redux overhead
Forms	React Hook Form + Zod	7.x / 3.x	Performant, schema-validated forms
Testing	Vitest + RTL + MSW	1.x	Fast, Vite-native test runner with API mocking
Linting	ESLint + Prettier	8.x / 3.x	Code consistency enforced via Husky pre-commit

Backend Stack

LAYER	TECHNOLOGY	VERSION	REASON
Runtime	Node.js	22 LTS	Async I/O, large npm ecosystem, matches TypeScript
Framework	Express.js	4.x	Minimal, flexible, production-proven REST framework
Language	TypeScript	5.3	Strict mode, no any types, shared types with frontend
ORM	Mongoose	8.x	Schema enforcement, middleware hooks, population
Validation	Zod	3.x	TypeScript-first, co-locate schema with route
Auth	JWT + bcrypt	—	Stateless tokens, HTTP-only cookie delivery
Real-time	Socket.IO	4.x	WebSocket with graceful fallback, room-based events
Queues	BullMQ	5.x	Redis-backed, retry logic, job scheduling
Caching	ioredis	5.x	Redis client with cluster/sentinel support
Logging	Winston + Morgan	3.x	Structured logs, request logging
Security	Helmet + express-rate-limit	7.x	HTTP headers, DoS protection
Testing	Jest + Supertest	29.x / 6.x	API integration tests, coverage reports

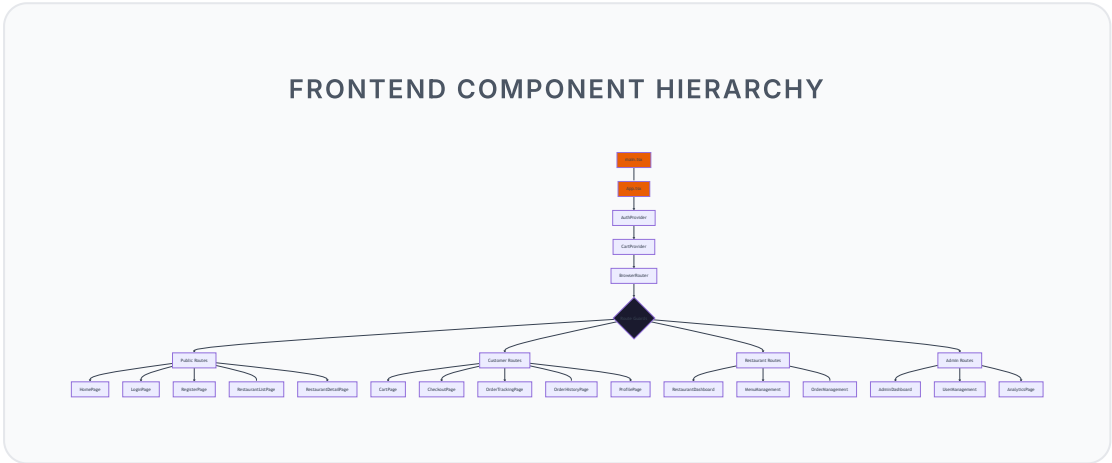
Infrastructure Stack

LAYER	TECHNOLOGY	REASON
Database	MongoDB Atlas M10+	Managed, auto-scale, backups, Atlas Search
Cache	Redis Cloud / Upstash	Managed Redis, serverless-friendly
Image CDN	Cloudinary	Transform on-the-fly, free tier, CDN delivery
Frontend Host	Vercel	Edge network, preview deployments, zero config
Backend Host	Render / Railway	Docker-based PaaS, auto-deploy from GitHub
Proxy	Nginx	SSL termination, gzip, load balancing
Containers	Docker + Compose	Dev/prod parity, local stack orchestration
CI/CD	GitHub Actions	Automated lint → test → build → deploy pipeline
Payments	Paytm Payment Gateway	Project requirement, UPI + card support

SECTION 04

Frontend Architecture

Component hierarchy, routing, state, and services



Application Pages

PAGE	ROUTE	ACCESS	KEY FEATURES
HomePage	/	Public	Hero, featured restaurants, search, cuisine filter
RestaurantListPage	/restaurants	Public	Filter by cuisine/rating/distance, map view
RestaurantDetailPage	/restaurants/:id	Public	Menu categories, add to cart, restaurant info
LoginPage	/auth/login	Public	Email+password, redirect post-login
RegisterPage	/auth/register	Public	Role selection, Zod validation
CartPage	/cart	Customer	Edit quantities, remove items, proceed to checkout
CheckoutPage	/checkout	Customer	Address selection, order summary, Payment
OrderTrackingPage	/orders/:id/track	Customer	Live status updates via Socket.IO
OrderHistoryPage	/orders	Customer	Past orders, reorder, review
ProfilePage	/profile	Customer	Edit profile, manage addresses
RestaurantDashboard	/dashboard	Owner	Today's orders, revenue, status toggle
MenuManagement	/dashboard/menu	Owner	CRUD menu items, upload images

PAGE	ROUTE	ACCESS	KEY FEATURES
AdminDashboard	/admin	Admin	Platform analytics, user/restaurant management

State Management

AuthContext

```
// src/context/AuthContext
interface AuthState {
  user: User | null;
  isLoading: boolean;
  isAuthenticated: boolean;
}
// Actions: LOGIN, LOGOUT,
```

CartContext

```
// src/context/CartContext
interface CartState {
  items: CartItem[];
  restaurantId: string | null;
  totalAmount: number;
  itemCount: number;
}
// Actions: ADD, REMOVE, U
```

Custom Hooks & Services

HOOK / SERVICE	FILE	PURPOSE
useAuth	hooks/useAuth.ts	Login, logout, register, getCurrentUser
useCart	hooks/useCart.ts	Add/remove items, sync with backend cart
useOrders	hooks/useOrders.ts	Fetch orders, track status, reorder
useRestaurants	hooks/useRestaurants.ts	Fetch list, nearby, search, by ID
useSocket	hooks/useSocket.ts	Socket.IO connection, order room events
authService	services/auth.service.ts	Axios calls to /api/auth/*
restaurantService	services/restaurant.service.ts	Axios calls to /api/restaurants/*
orderService	services/order.service.ts	Axios calls to /api/orders/*
paymentService	services/payment.service.ts	Initiate Paytm, poll status

SECTION 05

Backend Architecture

Request lifecycle, middleware stack, and module design

REQUEST LIFECYCLE — MIDDLEWARE PIPELINE



Middleware Stack (server.ts)

```
// backend/src/server.ts — Middleware registration order
app.use(cors({ origin: process.env.ALLOWED_ORIGINS, credentials: true }));
app.use(helmet()); // Security headers
app.use(express.json({ limit: '10mb' }));
app.use(cookieParser());
app.use(compression());

// Routes
app.use('/api/auth', authLimiter, authRouter); // 5 req/15min
app.use('/api/users', authenticate, userRouter);
app.use('/api/restaurants', restaurantRouter);
app.use('/api/cart', authenticate, cartRouter);
app.use('/api/orders', authenticate, orderLimiter, orderRouter);
app.use('/api/payments', paymentRouter); // webhook is pub
app.use('/api/reviews', authenticate, reviewRouter);
app.use('/api/admin', authenticate, authorize('admin'), adminRouter);
app.use(errorHandler); // Global error handler
```

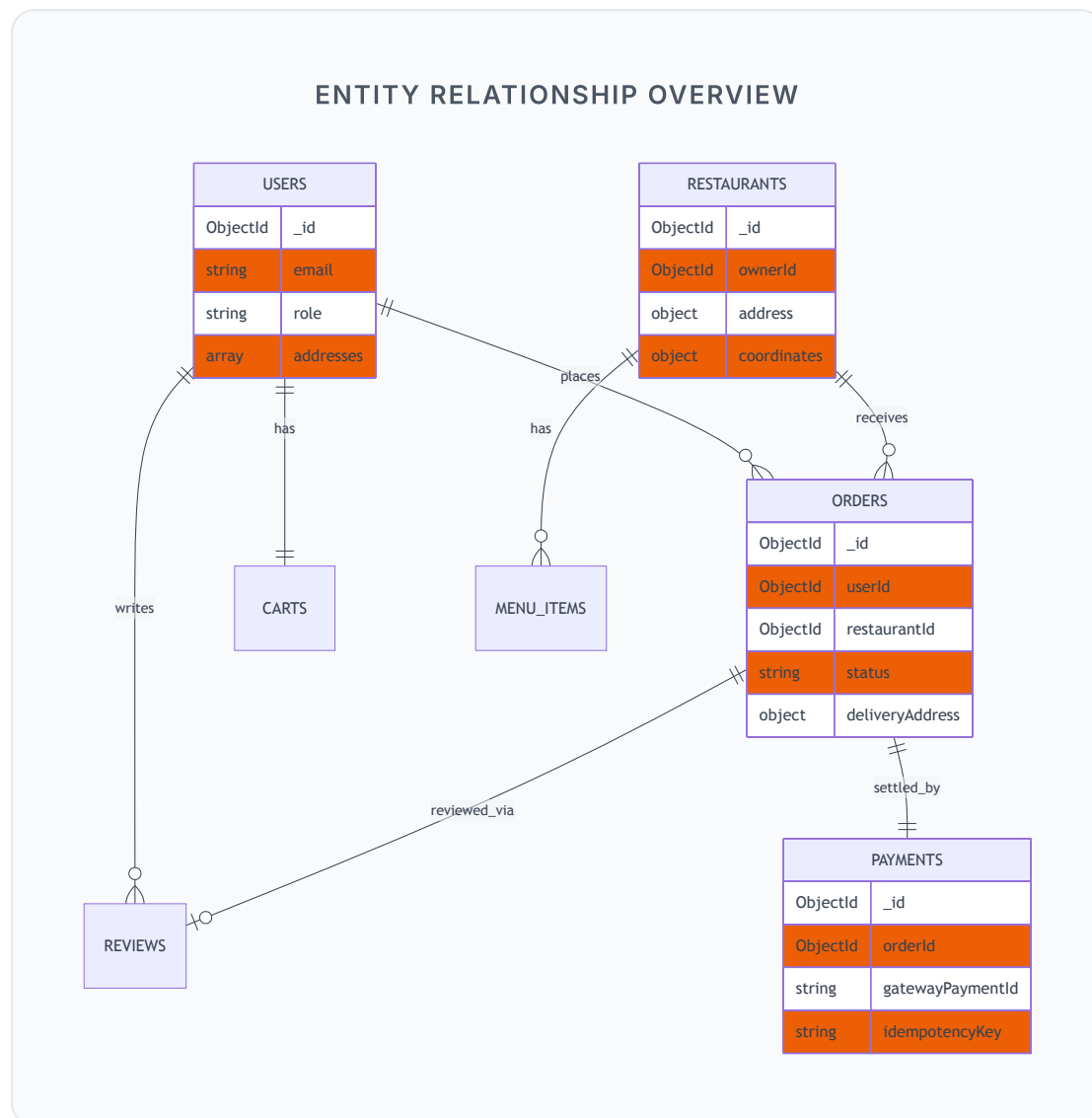
Module Breakdown

MODULE	CONTROLLER	SERVICE	ROUTES
Auth	auth.controller.ts	auth.service.ts	register, login, logout, refresh, /me
Users	user.controller.ts	user.service.ts	profile CRUD, address management
Restaurants	restaurant.controller.ts	restaurant.service.ts	CRUD, nearby, search, verify
Menu	menu.controller.ts	menu.service.ts	CRUD menu items, image upload
Cart	cart.controller.ts	cart.service.ts	get, add, update, clear
Orders	order.controller.ts	order.service.ts	create, get, update status, list
Payments	payment.controller.ts	payment.service.ts	initiate, webhook, status
Reviews	review.controller.ts	review.service.ts	create, list by restaurant
Admin	admin.controller.ts	admin.service.ts	users, restaurants, analytics

SECTION 06

Database Design (LLD)

MongoDB collections, schemas, indexes, relationships



Collection 1 — users

```
// Mongoose Schema
{
  _id:      ObjectId,
  name:     String (required, trim),
  email:    String (required, unique, lowercase),
```

```

passwordHash: String (required, select: false),
role:         'customer' | 'restaurant' | 'admin' (default: 'customer'),
phone:        String (optional),
avatar:       String (Cloudinary URL),
addresses: [{
  _id: ObjectId, label: String,
  street: String, city: String, state: String, pincode: String,
  coordinates: { lat: Number, lng: Number }, isDefault: Boolean
}],
isActive:     Boolean (default: true),
createdAt:    Date, updatedAt: Date
}
// Indexes: email (unique), role

```

Collection 2 — restaurants

```

{
  _id:          ObjectId,
  ownerId:      ObjectId (ref: 'User', required),
  name:         String (required, text-indexed),
  description:  String,
  cuisineTypes: [String] (indexed),
  logo:         String (Cloudinary URL),
  coverImage:  String (Cloudinary URL),
  address: {
    street: String, city: String, state: String, pincode: String,
    location: { type: 'Point', coordinates: [lng, lat] } // 2dsphere index
  },
  rating:       Number (default: 0, min:0, max:5),
  totalRatings: Number (default: 0),
  isOpen:       Boolean (default: false),
  openingHours: { mon-sun: { open: String, close: String } },
  deliveryRadius: Number (km),
  minOrder:     Number,
  deliveryFee:   Number,
  avgDeliveryTime: Number (minutes),
  isVerified:    Boolean (default: false),
  createdAt:    Date, updatedAt: Date
}
// Indexes: address.location (2dsphere), cuisineTypes, isOpen, name (text)

```

Collection 3 — menuitems

```

{
  _id: ObjectId,
  restaurantId: ObjectId (ref: 'Restaurant', required),
  name: String (required),
  description: String,
  price: Number (required, min: 0),
  category: String (e.g. 'Starters', 'Mains', 'Desserts'),
  image: String (Cloudinary URL),
  isVeg: Boolean (default: false),
  isAvailable: Boolean (default: true),
  tags: [String] (e.g. ['spicy', 'bestseller']),
  customizations: [{
    name: String, // e.g. "Size", "Toppings"
    options: [{ label: String, extraPrice: Number }]
  }],
  createdAt: Date, updatedAt: Date
}
// Indexes: restaurantId, category, isAvailable, { restaurantId, category

```

Collection 4 — carts

```

{
  _id: ObjectId,
  userId: ObjectId (ref: 'User', unique), // one cart per user
  restaurantId: ObjectId (ref: 'Restaurant'),
  items: [{
    menuItemId: ObjectId (ref: 'MenuItem'),
    name: String, price: Number,
    quantity: Number (min: 1),
    customizations: [{ name: String, option: String, extraPrice: Number }],
    subtotal: Number
  }],
  totalAmount: Number,
  updatedAt: Date
}
// TTL Index: updatedAt (expireAfterSeconds: 86400) - auto-clears abandoned carts
// Index: userId (unique)

```

Collection 5 — orders

```

{
  _id:           ObjectId,
  userId:        ObjectId (ref: 'User'),
  restaurantId:  ObjectId (ref: 'Restaurant'),
  items: [{
    menuItemId:  ObjectId, name: String, price: Number,
    quantity:    Number, customizations: [...], subtotal: Number
  }],
  deliveryAddress: { street, city, state, pincode, coordinates }, // s
  status:          'pending'|'confirmed'|'preparing'|'out_for_delivery',
  paymentStatus:   'pending'|'paid'|'failed'|'refunded',
  paymentId:       ObjectId (ref: 'Payment'),
  subtotal:        Number,
  deliveryFee:     Number,
  taxes:           Number,
  totalAmount:     Number,
  estimatedDelivery: Date,
  deliveryAgent:   String (optional),
  idempotencyKey:  String (unique),
  createdAt: Date, updatedAt: Date
}
// Indexes: userId, restaurantId, status, createdAt, idempotencyKey (un
// Shard Key: userId (for horizontal scaling)

```

Collection 6 — payments

```

{
  _id:           ObjectId,
  orderId:        ObjectId (ref: 'Order', unique),
  userId:         ObjectId (ref: 'User'),
  amount:         Number,
  currency:       'INR',
  gateway:        'paytm',
  gatewayOrderId: String (Paytm TxnToken),
  gatewayPaymentId: String (Paytm TxnId),
  checksum:       String,
  status:         'pending'|'success'|'failed'|'refunded',
  idempotencyKey: String (unique),
  metadata:       Mixed (raw Paytm response),
  createdAt: Date, updatedAt: Date
}
// Indexes: orderId (unique), gatewayPaymentId, idempotencyKey (unique)

```

Collection 7 — reviews

```
{
  _id:          ObjectId,
  userId:       ObjectId (ref: 'User'),
  restaurantId: ObjectId (ref: 'Restaurant'),
  orderId:      ObjectId (ref: 'Order', unique), // one review per order
  rating:       Number (min:1, max:5, required),
  comment:      String (max: 500),
  images:       [String] (Cloudinary URLs, max 3),
  helpfulCount: Number (default: 0),
  createdAt:    Date
}
// Indexes: restaurantId, userId, orderId (unique)
```

Indexing Strategy Summary

COLLECTION	INDEX	TYPE	PURPOSE
users	email	Unique	Login lookup
restaurants	address.location	2dsphere	Geo-proximity search
restaurants	{ cuisineTypes, isOpen }	Compound	Filter queries
restaurants	name	Text	Search autocomplete
menuitems	{ restaurantId, category }	Compound	Menu fetch by category
carts	userId	Unique	One cart per user
carts	updatedAt	TTL (24h)	Auto-expire abandoned carts
orders	{ userId, createdAt }	Compound	Order history
orders	idempotencyKey	Unique	Prevent duplicate orders
payments	orderId	Unique	One payment per order
payments	idempotencyKey	Unique	Payment retry safety
reviews	orderId	Unique	One review per order

SECTION 07

REST API Design

All endpoints, methods, auth requirements, and responses

Auth Endpoints — /api/auth

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	/api/auth/register	Public	Create account (customer/restaurant)
POST	/api/auth/login	Public	Login → set HTTP-only JWT cookie
POST	/api/auth/logout	JWT	Clear auth cookie
POST	/api/auth/refresh	Refresh Token	Issue new access token
GET	/api/auth/me	JWT	Get current user profile

Restaurant Endpoints — /api/restaurants

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/restaurants	Public	List all (paginated, filter: cuisine, rating, open)
GET	/api/restaurants/nearby	Public	Geo search: ? lat=&lng=&radius=
GET	/api/restaurants/search	Public	Full-text search: ? q=biryani
GET	/api/restaurants/:id	Public	Restaurant detail + menu
POST	/api/restaurants	Owner	Create restaurant profile
PATCH	/api/restaurants/:id	Owner	Update restaurant details
PATCH	/api/restaurants/:id/toggle	Owner	Toggle open/closed status
DELETE	/api/restaurants/:id	Admin	Remove restaurant

Menu Endpoints — /api/restaurants/:id/menu

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/restaurants/:id/menu	Public	Get all menu item (grouped by cate
POST	/api/restaurants/:id/menu/items	Owner	Add menu item (v image upload)
PATCH	/api/restaurants/:id/menu/items/:itemId	Owner	Update item details/price/avai
DELETE	/api/restaurants/:id/menu/items/:itemId	Owner	Remove menu ite

Cart Endpoints — /api/cart

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/cart	Customer	Get current user's cart
POST	/api/cart/items	Customer	Add item (validates same restaurant)
PATCH	/api/cart/items/:itemId	Customer	Update quantity
DELETE	/api/cart/items/:itemId	Customer	Remove item
DELETE	/api/cart	Customer	Clear entire cart

Order Endpoints — /api/orders

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	/api/orders	Customer	Place order (from cart, with idempotency key)
GET	/api/orders/my	Customer	Customer's order history (paginated)
GET	/api/orders/:id	Customer/Owner	Order detail
PATCH	/api/orders/:id/status	Owner/Admin	Update order status
POST	/api/orders/:id/cancel	Customer	Cancel order (before preparing)
GET	/api/orders/restaurant	Owner	Restaurant's incoming orders

Payment Endpoints — /api/payments

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	/api/payments/initiate	Customer	Generate Paytm transaction token
POST	/api/payments/webhook	Paytm Signature	Paytm payment callback (checksum verified)
GET	/api/payments/:id	Customer	Payment status + details

Review & Admin Endpoints

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	/api/reviews	Customer	Submit review (one per delivered order)
GET	/api/reviews/restaurant/:id	Public	Paginated reviews for restaurant
GET	/api/admin/users	Admin	All users with filters
GET	/api/admin/restaurants	Admin	All restaurants, verify/suspend
GET	/api/admin/analytics	Admin	Revenue, orders, active users stats
PATCH	/api/admin/restaurants/:id/verify	Admin	Verify restaurant for listing

Standard Response Format

```
// Success
{ "success": true, "data": { ... }, "message": "Order placed" }
```

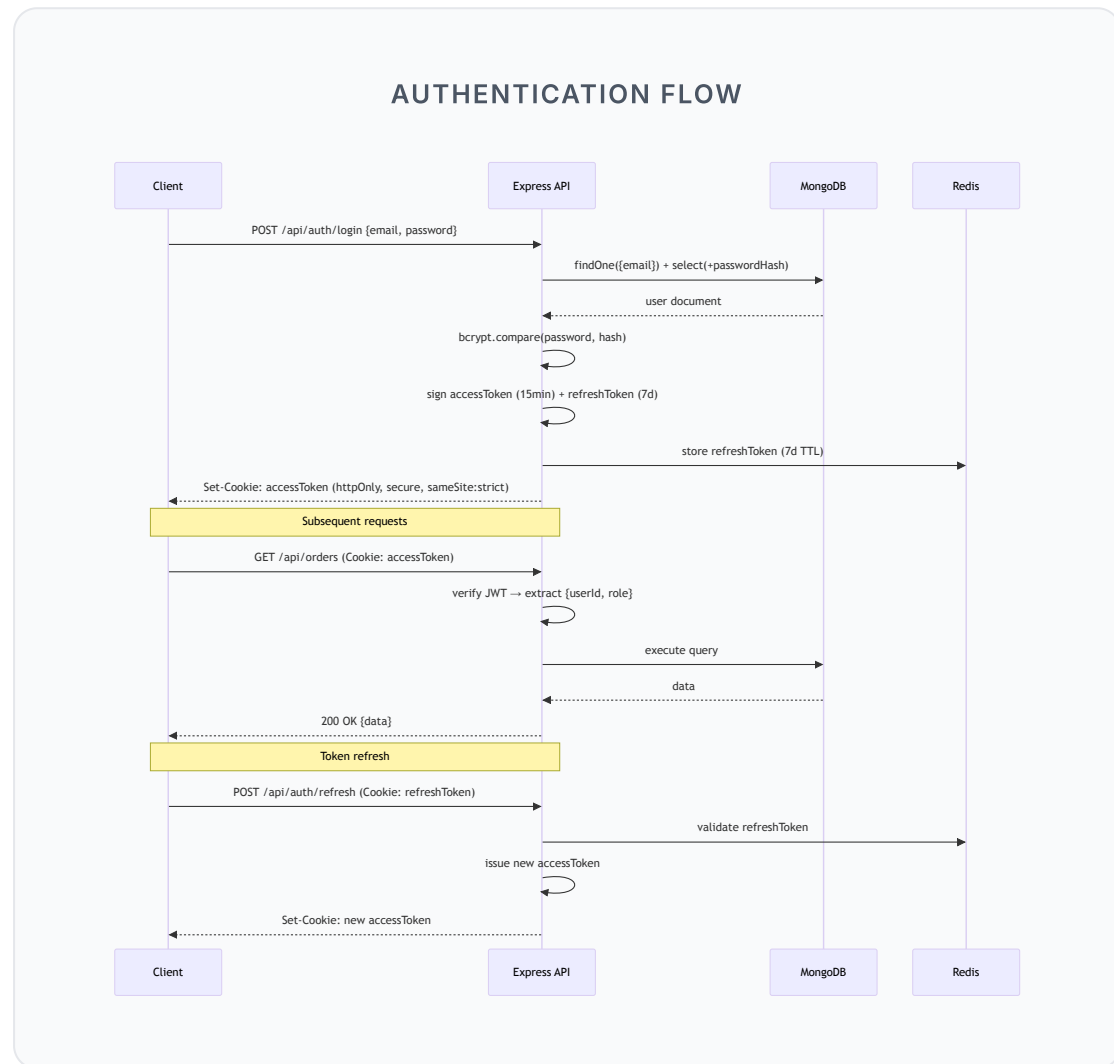
```
// Paginated
{ "success": true, "data": [...], "pagination": { "page": 1, "limit": 2

// Error
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "
```

SECTION 08

Authentication & RBAC

JWT flow, HTTP-only cookies, role-based access control



Role Permissions Matrix

ACTION	CUSTOMER	RESTAURANT OWNER	ADMIN
Browse restaurants & menu	✓	✓	✓
Add to cart / Place order	✓	✗	✗
View own orders	✓	✓	✓
Cancel own order	✓	✗	✓
Submit review	✓	✗	✗
Manage own restaurant	✗	✓	✓
CRUD menu items	✗	✓	✓
Update order status	✗	✓	✓
View all users	✗	✗	✓
Verify restaurants	✗	✗	✓
Platform analytics	✗	✗	✓

Middleware Implementation

```
// backend/src/middleware/auth.middleware.ts
export const authenticate = async (req, res, next) => {
  const token = req.cookies.accessToken;
  if (!token) return next(new AppError('Unauthorized', 401));
  const payload = jwt.verify(token, process.env.JWT_SECRET) as JwtPayload;
  req.user = { id: payload.userId, role: payload.role };
  next();
};

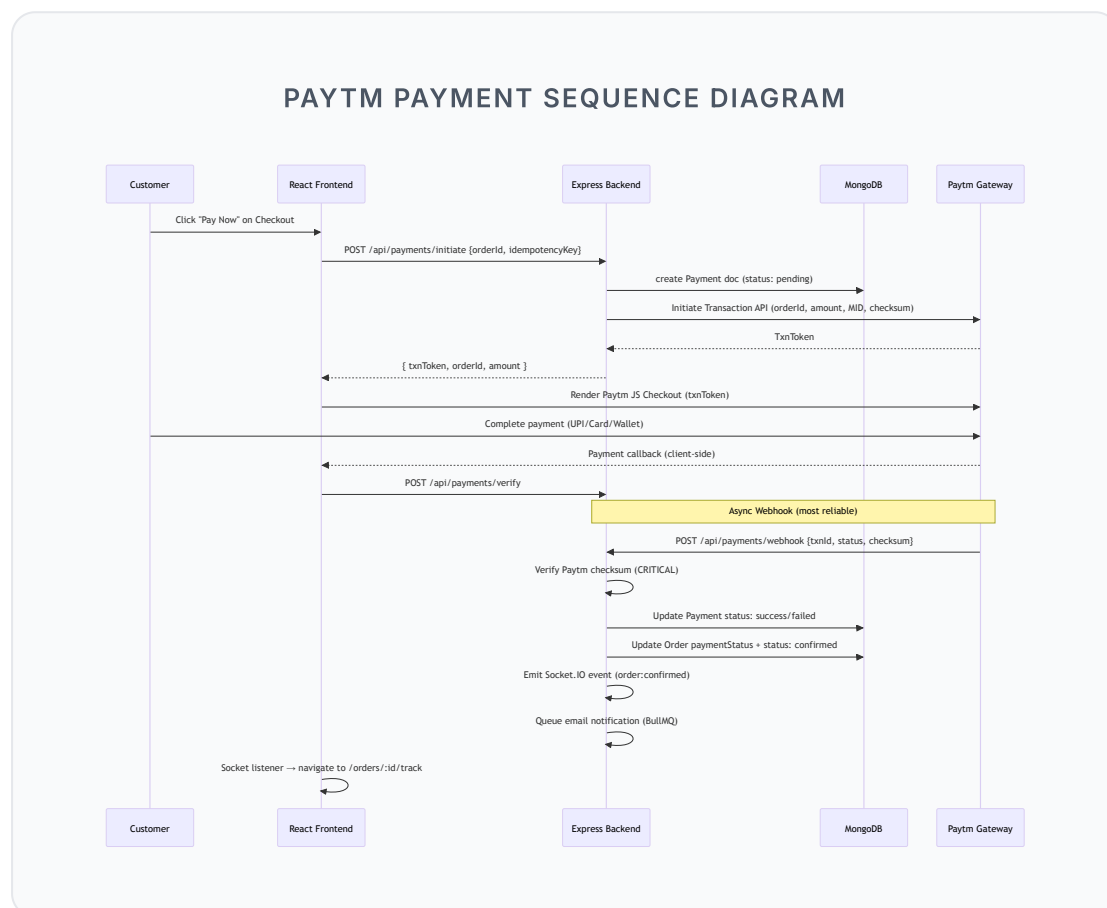
export const authorize = (...roles: Role[]) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return next(new AppError('Forbidden', 403));
  next();
};
```

```
    next();  
};
```

SECTION 09

Payment Architecture

Paytm integration — initiation, callback, webhook, idempotency



Idempotency & Retry Safety

Idempotency Key Pattern

```
// Generated client-side
const idempotencyKey =
  `${userId}-${orderId}-${`
```

Checksum Verification

```
// backend/src/services/paytm
import PaytmChecksum from 'paytmchecksum'

const isValid = await
```

```
// Backend: check before p  
const existing = await Pay  
  { idempotencyKey }  
);  
if (existing) return exist
```

```
PaytmChecksum.verifySign  
  body, MERCHANT_KEY, ch  
);  
  
if (!isValid)  
  throw new AppError('Inva
```



Critical Security Note

Always verify payment status via Paytm's Transaction Status API from the backend — never trust client-side callback parameters alone. The webhook route must not be behind authentication middleware but MUST verify the Paytm checksum on every call.

SECTION 10

Real-Time Order Tracking

Socket.IO events, rooms, and Redis Pub/Sub for scale

SOCKET.IO REAL-TIME ARCHITECTURE



Syntax error in text
mermaid version 11.12.3

Socket.IO Events

EVENT	DIRECTION	ROOM	PAYLOAD
<code>order:confirmed</code>	Server → Customer	order: {orderId}	{ orderId, status, estimatedDelivery }
<code>order:preparing</code>	Server → Customer	order: {orderId}	{ orderId, status }
<code>order:dispatched</code>	Server → Customer	order: {orderId}	{ orderId, status, deliveryAgent }
<code>order:delivered</code>	Server → Customer	order: {orderId}	{ orderId, status }
<code>order:new</code>	Server → Restaurant	restaurant: {id}	{ order full object }
<code>order:cancelled</code>	Server → Both	order: {orderId}	{ orderId, reason }
<code>join:order</code>	Client → Server	—	{ orderId }
<code>join:restaurant</code>	Client → Server	—	{ restaurantId }

```
// backend/src/services/socket.service.ts
export const emitOrderStatus = (
  orderId: string, event: OrderEvent, payload: OrderEventPayload
) => {
  // Publish to Redis - all instances receive and re-emit
  redisClient.publish('order-events', JSON.stringify({ orderId, event,
});

// Socket.IO setup
io.on('connection', (socket) => {
  socket.on('join:order', ({ orderId }) => socket.join(`order:${orderId}`)
  socket.on('join:restaurant', ({ restaurantId }) => socket.join(`restaurant:${restaurantId}`));
});
```

SECTION 11

Security Architecture

Defense in depth — transport, application, and data layer



Transport Layer

- HTTPS everywhere (TLS 1.3)
- Nginx: HSTS, SSL certificate (Let's Encrypt)
- Vercel: auto-HTTPS on all deployments
- HTTP→HTTPS redirect enforced



Application Layer

- Helmet: CSP, X-Frame-Options, HSTS
- CORS: whitelist allowed origins only
- JWT in HTTP-only, Secure, SameSite=Strict cookies
- Zod validation on all request bodies
- bcrypt rounds: 12 for password hashing



Rate Limiting

- General API: 100 req / 15 min
- Auth routes: 5 req / 15 min (brute force)
- Order creation: 10 req / min
- Backed by Redis (distributed)



Data Layer

- MongoDB Atlas IP whitelist
- Separate DB users per service (least privilege)
- Idempotency keys on payments
- Paytm checksum verification on webhook
- ObjectId validation on all params

Security Headers (Helmet Config)

```
// backend/src/config/helmet.config.ts
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self', 'https://securegw.paytm.in'],
      frameSrc: ['https://securegw.paytm.in'],
      imgSrc: ['self', 'https://res.cloudinary.com', 'data:'],
    },
  },
  hsts: { maxAge: 31536000, includeSubDomains: true },
  noSniff: true,
  xssFilter: true,
  frameguard: { action: 'deny' },
}));
```

Input Validation Pattern (Zod)

```
// backend/src/validators/order.validator.ts
export const createOrderSchema = z.object({
  body: z.object({
    deliveryAddressId: z.string().regex(/^a-f\d{24}$/i, 'Invalid Object'),
    idempotencyKey: z.string().min(20).max(100),
    paymentMethod: z.enum(['paytm']),
  })
});

// Applied as middleware
router.post('/', validate(createOrderSchema), orderController.create);
```

SECTION 12

Deployment Architecture

CI/CD pipeline, Docker, and cloud hosting

CI/CD PIPELINE — GITHUB ACTIONS



Docker Compose (Local Dev)

```
# docker-compose.yml
services:
  mongodb:
    image: mongo:7
    volumes: [mongo-data:/data/db]
    ports: ["27017:27017"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  backend:
    build: ./backend
    env_file: ./backend/.env
    ports: ["5000:5000"]
    depends_on: [mongodb, redis]
    volumes: [./backend/src:/app/src] # hot reload

  frontend:
    build: ./frontend
    env_file: ./frontend/.env
    ports: ["5173:5173"]
    depends_on: [backend]
```

```
nginx:
  image: nginx:alpine
  volumes: [./nginx.conf:/etc/nginx/conf.d/default.conf]
  ports: ["80:80", "443:443"]
  depends_on: [frontend, backend]
```

Environment Variables

VARIABLE	FILE	DESCRIPTION
MONGODB_URI	backend/.env	MongoDB Atlas connection string
REDIS_URL	backend/.env	Redis connection URL
JWT_SECRET	backend/.env	JWT signing secret (256-bit minimum)
JWT_REFRESH_SECRET	backend/.env	Separate secret for refresh tokens
PAYTM_MID	backend/.env	Paytm Merchant ID
PAYTM_MERCHANT_KEY	backend/.env	Paytm Merchant Key (never log)
CLOUDINARY_URL	backend/.env	Cloudinary API URL
ALLOWED_ORIGINS	backend/.env	Comma-separated CORS origins
VITE_API_URL	frontend/.env	Backend API base URL
VITE_PAYTM_MID	frontend/.env	Paytm MID for JS checkout

SECTION 13

Scalability Design

Caching, queues, sharding, and horizontal scaling

Redis Caching Strategy

CACHED DATA	CACHE KEY	TTL	INVALIDATION
Restaurant list (by city)	restaurants:city:{city}	5 min	On restaurant update/create
Menu items	menu:restaurant:{id}	10 min	On menu item CRUD
Restaurant detail	restaurant:{id}	10 min	On restaurant update
User session	session:{userId}	7 days	On logout
Rate limit counter	rl:{ip}:{route}	15 min	Auto-expire

BullMQ Job Queues

QUEUE	JOB	RETRY POLICY	PRIORITY
order-processing	validateOrder, notifyRestaurant, clearCart	3 retries, exp backoff	High
notification	pushNotification, smsAlert	2 retries	Medium
email	orderConfirmation, passwordReset, receipt	5 retries, 1hr delay	Low
analytics	updateRestaurantRating, dailyReport	1 retry	Low

MongoDB Sharding (Production Scale)

Orders Collection

```
// Shard by userId
sh.shardCollection(
  "magizhthunnu.orders",
  { userId: "hashed" }
)
// Distributes load evenly
// Co-locates a user's orders
```

Restaurants Collection

```
// Shard by location
sh.shardCollection(
  "magizhthunnu.restaurant",
  { "address.location": "2"
}
)
// Geo-local queries fast
// City-level partitioning
```

Load Balancing (Nginx)

```
# nginx.conf - upstream for multiple backend instances
upstream backend {
    least_conn;                # route to least busy
    server backend1:5000;
    server backend2:5000;
```

```
server backend3:5000;
    keepalive 32;
}

server {
    listen 443 ssl;
    location /api { proxy_pass http://backend; }
    location /socket.io { proxy_pass http://backend; proxy_http_version 1
        proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection
    }
```

SECTION 14

Testing Strategy

Unit, integration, and E2E testing with 80%+ coverage target

Frontend Testing

- **Vitest** — Unit tests for hooks, utils, services
- **React Testing Library** — Component interaction tests
- **MSW (Mock Service Worker)** — API response mocking
- **Coverage target:** 80%+

Backend Testing

- **Jest** — Unit tests for services, utils, validators
- **Supertest** — HTTP integration tests for all routes
- **mongodb-memory-server** — In-memory DB for tests
- **Coverage target:** 80%+

Critical Test Cases by Module

MODULE	TEST CASES
Auth	Register with duplicate email (400), login with wrong password (401), JWT expiry → 401, refresh token flow
Cart	Add items from different restaurants (400), quantity update, cart auto-clear on checkout
Orders	Place order from empty cart (400), idempotency key duplicate (409), status transition validation
Payments	Invalid Paytm checksum rejected (400), duplicate idempotency key returns cached, failed payment doesn't confirm order
RBAC	Customer cannot update order status (403), owner cannot access another restaurant (403), admin can access all
Rate Limiting	5+ auth attempts within 15min → 429, order burst → 429
Reviews	Review on undelivered order (400), duplicate review on same order (409)

Test Command Reference

```
# Backend
npm test                # Run all Jest tests
npm run test:coverage    # Coverage report
npm run test:watch       # Watch mode

# Frontend
npm test                # Run Vitest
npm run test:ui          # Vitest UI
npm run test:coverage    # Coverage report
```

Complete Folder Structure

Frontend Structure

44/45

Backend Structure

```

backend/ └─ src/ │ └─ server.ts # Express app bootstrap +
middleware registration │ └─ config/ │ │ └─ db.ts # Mongoose
connection + retry logic │ │ └─ redis.ts # ioredis client setup │
│ │ └─ cloudinary.ts # Cloudinary SDK config │ │ └─ logger.ts #
Winston logger config │ └─ models/ # Mongoose schemas │ │ └─
User.model.ts │ │ └─ Restaurant.model.ts │ │ └─
MenuItem.model.ts │ │ └─ Cart.model.ts │ │ └─ Order.model.ts │ │
└─ Payment.model.ts │ │ └─ Review.model.ts │ └─ routes/ │ │ └─
auth.routes.ts │ │ └─ user.routes.ts │ │ └─ restaurant.routes.ts
│ │ └─ menu.routes.ts │ │ └─ cart.routes.ts │ │ └─
order.routes.ts │ │ └─ payment.routes.ts │ │ └─ review.routes.ts
│ │ └─ admin.routes.ts │ └─ controllers/ # Request handlers -
call services, send response │ └─ services/ # Business logic - DB
queries, external calls │ │ └─ payment.service.ts # Paytm
checksum, initiate, verify │ └─ middleware/ │ │ └─
auth.middleware.ts # authenticate + authorize │ │ └─
validate.middleware.ts # Zod request validation │ │ └─
rateLimiter.ts # Redis-backed rate limiter │ │ └─ errorHandler.ts
# Global error handler │ └─ queues/ │ │ └─ email.queue.ts │ │
└─ notification.queue.ts │ │ └─ orderProcessing.queue.ts │ └─
validators/ # Zod schemas per module │ └─ utils/ │ │ └─
AppError.ts │ │ └─ asyncHandler.ts # try/catch wrapper for async
routes │ │ └─ pagination.ts │ └─ types/ # Backend TypeScript
interfaces │ └─ tests/ │ └─ unit/ │ └─ integration/ │ └─ .env #
All backend secrets (never commit) │ └─ Dockerfile │
tsconfig.json # strict: true, no any └─ package.json

```

Magizhthunnu

System Design Document v1.0 — March 2026

Designed by Senior Full Stack Architect
MERN Stack · TypeScript · MongoDB · Paytm